Quick quiz

What is an agent composed of? Pick one.

1 Multiple LLM models working in parallel with distributed computing resources

2 A database system with query optimization capabilities

3 An LLM combined with a harness ✓

4 A neural network trained on specific task data

Click an answer. The other three are real things — none of them is an agent.

Quick quiz, again

Two products, same model underneath. One is better. Why? Pick one.

1 The speed of model inference

2 The choice of the latest foundational model

3 The quality of the harness implementation ✓

4 The volume of training data used

Everyone can rent the same model. **Almost nobody has the same harness.**
That is where the difference lives — and it is the whole reason this talk exists.

Quick quiz — what the loop is for

What is an agent loop actually doing? Pick one.

1 Storing tool results in a database

2 Running one tool call and returning the result straight away

3 Validating the format of user input

4 Going round again — calling tools until the model has enough to answer confidently ✓

Two is the one to argue about: that is a **function call**, not an agent. The loop is the difference — the model gets to look at what came back and decide it needs something else.

Quick quiz — why that loop is not enough

It runs. Why can you not ship it? Pick one.

1 It can only work with one tool at a time

2 It processes data too slowly

3 It needs too much memory

4 No error handling, nobody approves the risky bits, and nothing controls it ✓

Notice what is **not** on the list: speed, memory, and how many tools it can hold.
`Nothing wrong with that loop is a performance problem.`

Quick quiz — the step limit

Why does an agent workflow need a maximum number of steps? Pick one.

1 To cap the characters in each message

2 To stop the loop running away and burning time and money ✓

3 To cap how many tools can be registered

4 To count tokens

Remember the very first loop we looked at: `while True`, with nothing to stop it. `This is that bound.`

Quick quiz — no tool calls

The model comes back with zero tool calls. What does that mean? Pick one.

1 The step limit has been reached

2 Something errored and needs logging

3 It needs to retry

4 It has nothing left to do — this is the answer, and the loop is finished ✓

This is the **one** clean way out of the loop. Everything else that stops it is a failure wearing a result.

Quick quiz — the failure event

Why emit a "workflow failed" event when the loop exits? Pick one.

1 For debug logging

2 To record that it hit the step cap or errored — it did not finish ✓

3 To fail over to a different model provider

4 To restart the agent

Someone has to be told the difference between **done** and **stopped**. `If nothing emits that, a cut-off run looks exactly like a finished one.`

Quick quiz — the loop

What is the runtime file for? Pick one.

1 To store API keys and configuration

2 To manage database connections and long-term storage

3 To define the available AI models

4 The core environment where the agent runs — a game loop, for an agent ✓

Careful with the word: in **this** deck "the runtime" has meant the whole harness.
In many codebases it is also the name of the one file holding the loop.

Quick quiz — the unfinished conversation

You resend a message list that ends mid-exchange — a tool call with no result yet. What happens? Pick one.

1 **It is rejected — an unanswered tool call is a malformed list** ✓

2 The reply comes back unchanged

3 It carries on from where the exchange stopped

4 The conversation starts again from the top

Resuming really is just handing the list back — `but only a list that closes every exchange it opened.` Which is why you save the tool result, not the request that is still waiting for one.

Quick quiz — tool calling

What is tool calling? Pick one.

1 **A built-in way for a model to reach external functions and APIs** ✓

2 A logging system

3 A debugging feature

4 A performance optimisation for multi-region systems

This is how a model stops being a text box. It is also why **pattern 3** exists — the moment it can call things, something has to decide what it is allowed to call, and run it somewhere safe.

Quick quiz — the tool description

What is the description field in a tool definition for? Pick one.

1 Validating input parameters

2 Generating API documentation

3 Helping the model decide when to call the tool ✓

4 Documentation for the developers who maintain the code later

Four is the tempting one, and it is why so many tool descriptions are useless.
`The description is written for the model, not for you.`

Quick quiz — the input schema

And what is the input schema for? Pick one.

1 Encrypting sensitive data

2 A contract for the arguments the agent has to supply when it calls the tool ✓

3 Compressing data

4 Generating user interface components

Description says **when** to call it. Schema says **what has to be in the call**.
Between them, most bad tool calls stop being possible.

Quick quiz — describing the fields

Why bother describing each field in the schema? Pick one.

1 Documentation for humans only — models cannot read them

2 So CSS frameworks can style the form

3 To tell the agent what the field means, when the name alone does not ✓

4 To make the file bigger

A model will guess `item_id`. It will not guess a name your company uses oddly, or a value it has to fetch from somewhere else first. `Describe those.`

Quick quiz — what is in a tool call

The model asks for a tool. What comes back? Pick one.

1 **A call id, the tool name, and the arguments** ✓

2 Just a call id

3 Id, name, arguments and an execution timestamp

4 Just the name and the arguments

Three is the tempting one: there is no timestamp, because **nothing has executed yet**. This is a request, not a record. The `timestamp is yours to add, when you run it.`

Quick quiz — where the tool runs

The model asks for a tool call. Where does it actually run? Pick one.

1 In the browser

2 In cloud storage

3 On the model provider's servers, automatically

4 On your own server, after the model asks for it ✓

Three is the belief worth correcting today. The `provider has no copy of your code.` And that is good news — it is why a guardrail or a human gate can exist at all.

Quick quiz — the call id

Why does a tool call carry an id? Pick one.

1 To authenticate the tool with the provider

2 To encrypt the message

3 So the result can be matched back to the request that asked for it ✓

4 To store tool calls for auditing

Send several calls at once and the answers come back out of order. The id is how anything makes sense of that. `Drop it and most models break outright.`

Quick quiz — sending a result back

You ran the tool. Which role carries the result back to the model? Pick one.

1 user — it is going into the conversation

2 assistant — it belongs with the call that asked

3 tool — with the id of the call it answers ✓

4 system — it is machine output

The roles are not decoration. `Put a tool result in as the user and you have just told the model a person said it` — which is exactly the confusion pattern 1 exists to prevent.

Quick quiz — what has to be last

You have built the list and you are about to call the model. What must be on the end of it? Pick one.

1 Something waiting on an answer — the person's message, or a tool result ✓

2 The standing instructions, repeated at the bottom

3 An empty message, to show the turn is over

4 A sample reply, to show it the format you want

The list is a question, and the last line is the question. End on the model's own previous reply and you have handed it **nothing to respond to**. `Standing instructions go at the top and stay there` — putting them last does not make them louder.

Quick quiz — the roles

What are the four roles in a message list? Pick one.

1 prompt, response, context, function

2 admin, client, server, callback

3 input, output, error, response

4 system, user, assistant, tool ✓

The whole conversation is just that list, replayed every turn. `Which is why it costs what it costs.`

Quick quiz — the message list

Why append every reply back onto the list? Pick one.

1 The model keeps no memory of previous turns — you resend everything, every time ✓

2 To make later queries faster

3 To reduce API costs

4 To make streaming work

Two and three are not just wrong, they are **backwards**: a longer list is slower and dearer. `You pay for the memory, every single turn.`

Quick quiz — as the list grows

The conversation gets longer. What happens? Pick one.

1 Nothing changes

2 It costs more, but runs just as fast

3 It costs more *and* gets slower ✓

4 It improves — the model learns from the history

Four is the comfortable myth. Nothing is being learned; the same text is simply being read again, from scratch, on every turn. `That is what you are paying for.`

Quick quiz — streaming

What does a streaming text call do? Pick one.

1 Sends the messages and streams the reply back token by token ✓

2 Stores the conversation in a database

3 Compresses data

4 Validates the input before it reaches the model

It is not faster. The words arrive at the same rate either way — but a blank screen for twenty seconds reads as a crash, and streaming lets someone stop a bad answer early.

Quick quiz — waiting for the stream

Why wait for the stream to finish before appending? Pick one.

1 Browser compatibility

2 To prevent memory leaks

3 To optimise the connection

4 So you append the finished message, not half of one ✓

A half-streamed reply pushed into the history is not a rendering glitch. The next turn reads it as something the agent actually said.

Quick quiz — swapping model providers

How does the Vercel AI SDK support multiple providers? Pick one.

1 **A separate module per provider, all working with the same methods like streamText or generateText** ✓

2 Runtime configuration files edited by hand for each switch

3 Separate API calls per provider

4 A universal API key

Different ecosystem, **the same trick worth stealing**: one agreed shape, many implementations behind it. Change the import, not the code that calls it. We do it with a `Protocol`; they do it with a module per provider.

Quick quiz — the system prompt

What is a system prompt? Pick one.

1 A debugging tool that traces the agent's decisions

2 The error shown when the agent fails

3 The user's question that starts the conversation

4 The standing instructions that define how the agent behaves ✓

Three is the near-miss worth naming: that is the **user** message. The system prompt comes from whoever **built** the agent, and the user never sees it.

Quick quiz — idempotency

Why must a step be safe to run more than once? Pick one.

1 It compresses data

2 It reduces latency

3 So running it twice gives the same result, with no second side effect ✓

4 It scales the system horizontally

"Work out the total" twice is harmless. **"Send the money" twice is not.** The `save comes after the step, so that gap is real.`

Quick quiz — you already use one

What makes CI/CD a distributed system you already know? Pick one.

1 **It runs elsewhere, talks back to other systems, and updates status whether or not you are watching** ✓

2 It uses AI to write the tests

3 It needs someone to press the buttons

4 It only runs locally

Stages you can retry. Caching between them. Status that arrives whether the tab is open or not. Recovery when a runner dies. `You have been using durable execution for years.`

Quick quiz — where the save points live

Durable-execution tools exist to do what? Pick one.

1 Handle authentication

2 Replace the database with an in-memory cache

3 Compile code faster

4 Keep every step's inputs and outputs in a database, so a run can resume ✓

Wrap an operation in a **step** and its inputs *and outputs* are recorded — so a replay **skips** it rather than redoing it. Several tools do this — Temporal, Inngest, DBOS — and **the Agent Framework has its own, which is what this repo runs**. `Ours is a seam`: a file on a laptop today, a cloud database later.

Quick quiz — the client goes away

Someone closes the tab mid-conversation, then comes back. What should happen? Pick one.

1 Throw the context away, for security

2 Pick up where they left off ✓

3 Start again from the beginning

4 Wait for someone to intervene

The work does not belong to the connection. On reconnect you **replay the log** to them — which is what the sequence numbers were for — and they catch up to wherever the run got to. `This is the crash demo again`, except nothing crashed; someone just left.

Quick quiz — what deserves a save point?

You cannot save after every line. Which of these is worth wrapping? Pick one.

1 Adding up numbers you already have in front of you

2 Anything that reaches out of your process — slow, unrepeatable, or paid for ✓

3 Every line, to be safe

4 Only writes to the database

Three questions, and **any yes means wrap it**: does it **leave your process** and take time? Would it **answer differently** if you ran it again? Did it **cost money**? `Pure local arithmetic needs none of it` — a replay will work it out again, identically, for nothing.

Quick quiz — making a step durable

How do you turn an ordinary operation into a recorded step? Pick one.

1 Add it to a workflow configuration object first

2 Register it up front, before the workflow runs

3 Create it with a name and a handler

4 Wrap the operation, passing the work in as a function ✓

Learn the **shape**, not the spelling. Wrapping means the library sees the call happen, so it can record what went in and what came out. `Registering it somewhere else could not.`

Quick quiz — the model call in a step

Why wrap the call to the model itself in a step? Pick one.

1 **So a restart does not buy the same expensive call twice** ✓

2 To turn a synchronous call asynchronous

3 To make the model faster by caching in memory

4 To run several model calls in parallel

Right — and there is a second reason that matters more. The model is **not deterministic**: ask again and it may answer differently and send the run down another path. `Recording the answer is what makes a replay a replay.`

Quick quiz — emits in a step

Why put the emit inside a step rather than loose in the workflow? Pick one.

1 To broadcast events in real time

2 So a restart cannot fire it a second time ✓

3 To improve performance

4 To get database transaction guarantees

An emit left loose in the workflow is **not recorded**, so a resume runs it again — and now the log says a thing happened twice that only happened once.

`Anything with an effect belongs in a step.`

Quick quiz — awaiting a streaming step

You wrap a streaming call in a step and await it. Does the streaming stop? Pick one.

1 Yes — awaiting collects it all before anything is shown

2 No — the stream still runs inside; you are waiting for the step to finish ✓

3 Only if the database is slow

4 Streaming cannot be used in a durable workflow

You are awaiting **the step**, not the first token. Tokens flow to the reader exactly as before; the step simply records what happened once it is done.

Quick quiz — when do you need a sandbox?

Two situations make it unavoidable. Which pair? Pick one.

1 Long conversations, and large files

2 When it runs risky code — shell, SQL — and when it writes programs that chain tools together ✓

3 When there are many users, and when latency matters

4 Only when handling payments

A coding agent running shell. An analysis agent running SQL. And **code mode**, where it writes a program instead of picking from a menu. `A menu is a bounded set of asks. A program is not.`

Quick quiz — why code mode wins

What does writing code buy over calling tools one at a time? Pick one.

1 **It combines several tools into one execution, so a dozen turns become one** ✓

2 It removes the need for a model at all

3 It generates documentation and an audit trail

4 It needs fewer logins

Sequential calling is ask, wait, think, ask again — every hop costing a round trip and a pile of tokens. `A program does the whole errand and comes back once.`

Quick quiz — who needs to run code?

Which agents actually benefit from being able to execute code? Pick one.

1 Ones handling logins, to run security checks

2 Ones doing analysis or writing software — real computation ✓

3 Ones that only summarise text

4 Ones generating images

If the job is **working something out**, code earns its place. If the job is producing prose, it is a liability you did not need. `Do not hand out a capability the task never asked for.`

Quick quiz — arithmetic

Why let the agent write code for a calculation instead of working it out? Pick one.

1 So it can avoid using any maths libraries

2 So it learns maths by trial and error

3 Code is deterministic — it gets the same right answer every time ✓

4 It halves the memory footprint

A model doing arithmetic is **predicting** what the answer looks like. Code is **computing** it. **Dates are the worst case** — "how many working days between these two" reads as a language question and is not one. `For anything that has one correct answer, do not ask a guesser.`

Quick quiz — why dates in particular

A model is worse at dates than at almost any other everyday thing. Why? Pick one.

1 Date formats vary from country to country

2 It never sees the date as a date — it gets chopped into fragments that mean nothing on their own ✓

3 It has never been shown a calendar

4 Too many digits, and digits cause mistakes

A model reads text in chunks, and `2026-08-25` may arrive as pieces like `202`, `6`, `-08`. **Nothing in there says "August", or "a Tuesday", or "31 days".**
`It is reassembling a calendar from crumbs, every time.`

Quick quiz — a bigger window

Your conversation keeps growing. You move to a model with a huge context window. Problem solved? Pick one.

1 Yes — room for everything, so nothing needs managing

2 No — it gets slower, costs more, and quietly loses the middle ✓

3 Yes, as long as you stay under the limit

4 No, because long conversations are refused outright

You resend the **whole** conversation every turn, so it gets slower and dearer with every message. Worse, attention is not even: **the start and the end land, the middle fades.** `A bigger window buys you room, not reliability` — which is why you summarise, and keep the bulk in a store you query.

Quick quiz — what you are allowed to drop

The history is too long, so something has to go. What is the smallest piece you can safely remove? Pick one.

1 Any single message — oldest first

2 A whole turn — the request and every tool result it was waiting on ✓

3 Any two messages, to keep the count even

4 Half the list, whenever it gets too long

A tool call and its result are joined by an id, and the result must come **immediately after** — nothing may be slipped in between. **Cut between them and you leave a question with no answer** — most models reject the list outright, and the ones that do not get confused. So you group the messages into turns, add up the whole lot to see if you are over, and roll up entire turns.

Quick quiz — the knowledge base

What is a knowledge base for? Pick one.

1 Logging every API call

2 Caching responses

3 A body of information the agent looks things up in, instead of carrying it all in the prompt ✓

4 Storing credentials and tokens

A wiki it **queries**, not a wiki it pastes. Paste it and every single turn costs you the whole corpus in tokens — for the three paragraphs it actually needed.

Quick quiz — where a handoff has to happen

You give the agent a “hand over to billing” tool. It calls it. What actually happens? Pick one.

1 Billing takes over the conversation

2 Nothing is handed over — the answer lands back in the same agent, which is still the one talking ✓

3 Both agents reply from then on

4 The conversation ends

Record **who it went to, who it came from, and why** while you are there — a transfer with no reason attached is the one nobody can explain a week later. A tool result always comes back **to the agent that asked** — that is the whole shape of the loop. **Nothing can take itself out of its own conversation and put something else there.** `Only the layer running the agents can change which one is running`, so the tool call is a *request* to hand over, and the runtime does it.

Quick quiz — how many agents do you need?

You have a genuinely hard problem. How many agents should work on it? Pick one.

1 One per step, so each stays simple

2 Usually one — a good model with a well-built harness around it ✓

3 At least three, or there is nothing to route between

4 One per tool

A crowd of agents is the exciting answer and rarely the right one. When you *do* split, the reason is usually not cleverness: **a destructive tool that only one agent may touch, different permissions, or different teams owning different parts.** `Organisational reasons, mostly — not capability ones.`

Quick quiz — who decides it needs approving?

A refund over £1,000 must be signed off. Where does that rule live? Pick one.

1 In the instructions — tell the model to ask before big refunds

2 In your code, as a plain rule that runs every time ✓

3 Give it an “ask a human” tool and let it choose when

4 Whichever is easier — they amount to the same thing

One and three both leave the gate to a **judgement call**, and a model that is right ninety-nine times in a hundred still lets one through. **An *if* statement is right every time.** Never make the safety of a system depend on the model remembering to be safe.

Quick quiz — the send button

You give an agent a "send reply" tool and no gate. What is the risk? Pick one.

1 It crashes under too many concurrent sends

2 It gets slower as the queue grows

3 It emails real people, in your name, before anyone has read the drafts ✓

4 It costs more in API calls

And it is not misbehaving when it does. `You told it to reply to the email; sending is how replying finishes.`

Quick quiz — the sequence number

Why does every row in an event log carry a sequence number? Pick one.

1 To cache frequently accessed data

2 To hold the database authentication token

3 To track the order of events, for replay and for spotting duplicates ✓

4 To encrypt the event

Events arrive late, and sometimes twice. `That is normal, not a bug you can engineer away` — so the log carries the order itself.

Quick quiz — save, then shout

When does emit tell the connected clients? Pick one.

1 **After the event is safely written to the database** ✓

2 Immediately, as soon as emit is called

3 Before the write, so listeners still get it if the write fails

4 Only on restart

Three sounds generous and is the dangerous one. `Tell people something happened, then fail to record it, and nobody can ever reconcile the two.`

Quick quiz — unsubscribing

What does subscribe hand back, and why? Pick one.

1 The last event, so you can catch up

2 A function that removes your listener — call it or they pile up ✓

3 A connection id for logging

4 Nothing

Every listener you add and never remove is a leak. In a process that runs for weeks, that is how you end up rebooting it every Tuesday.

Quick quiz — the payload column

Why hold the event's data as JSON in the database? Pick one.

1 It validates the payload against a schema on the way in

2 Every event carries a different shape, and the database can store JSON directly ✓

3 It encrypts the column

4 It compresses long strings

The columns you **query** — who, what kind, when, which number — get real columns. The bit that differs per event goes in one JSON field. `It does not validate anything.`

Quick quiz — the transport

Why WebSockets rather than server-sent events? Pick one.

1 They go both ways, so a distributed layer can push results to any connected client ✓

2 They are required by the SDK

3 They are faster than HTTP

4 They use less bandwidth

Speed and bandwidth are not the reason. **Direction** is: server-sent events only flow one way, so the client could never fire a task and be told about it later.

Quick quiz — emit

What does the emit pattern accomplish? Pick one.

1 It compresses data for transmission

2 It establishes secure connections between servers across regions

3 It tells anyone interested that an event has occurred ✓

4 It validates input parameters

And crucially it does **not wait** for anyone to answer. That is what stops one slow listener from holding up the work.

Quick quiz — is anyone still there?

Before writing to a socket you check its ready state. What are you checking? Pick one.

1 Whether the client is an administrator

2 Whether the connection is still open ✓

3 Whether the client is authenticated

4 Whether encryption is on

Clients leave without saying goodbye — closed laptops, dead wifi, shut tabs. `Writing into a socket nobody is holding is how a background job takes the server down with it.`

Quick quiz — now apply it

You are building an agent that trades money. What matters most? Pick one.

1 Chat interface design

2 Self-healing, guardrails, approval steps, and keeping financial data safe ✓

3 Real-time market charts

4 File upload

Every wrong answer is about the **screen**. The right one is the **harness** — and it is patterns 1, 6 and 7 with money on the line.